

EduAdapt: Adaptive Learning Engine

Permanent Project Reference & Verification Document

1. Short Project Summary

EduAdapt is a deterministic, AI-enhanced adaptive learning platform designed to solve the "one-size-fits-all" problem in traditional education. Instead of forcing all students down a linear curriculum, EduAdapt uses a diagnostic assessment to construct a personalized knowledge map.

The core **Adaptive Engine** acts as the source of truth, calculating mastery levels, identifying exact conceptual knowledge gaps, and dynamically recalculating the student's learning path based on prerequisite graphs. An integrated **Socratic AI Tutor** (powered by Gemini) provides conversational guidance, but strictly adheres to the engine's deterministic curriculum to prevent AI hallucinations from derailing the learning objectives.

Finally, the **Teacher Intelligence** module brings humans into the loop, aggregating student misconceptions across the classroom and empowering teachers to override the AI's recommendations when human intuition dictates a different intervention.

2. Exact Run Commands

The repository is structured as a monorepo with `client` (Next.js) and `server` (Express/Node.js).

Dependency Installation

```
# In Terminal 1
cd server
npm install
```

```
# In Terminal 2
cd client
npm install
```

Backend Startup

```
cd server
npx tsx src/server.ts
```

(The backend relies on `tsx` for on-the-fly TypeScript execution in development).

Frontend Startup

```
cd client
npm run dev
```

Seed Data

```
cd server
npx tsx src/utils/seed.ts
```

(Clears the database and inserts the baseline Subject, Topics, Questions, and the SIH Demo Student).

Testing Commands

The following programmatic integration/E2E tests exist in `server/src/utils/` :

```
cd server
npx tsx src/utils/test-loop.ts           # Tests the core adaptive loop
npx tsx src/utils/test-diagnostic.ts     # Tests the diagnostic assessment flow
npx tsx src/utils/test-tutor.ts         # Tests the AI Socratic Tutor
npx tsx src/utils/test-teacher.ts       # Tests Teacher RBAC and Overrides
npx tsx src/utils/test-e2e.ts           # Complete Master E2E Simulation
```

(Note: There are no standard `npm test` scripts mapped in `package.json` . Tests must be executed directly via `tsx`).

3. Environment Variables

The following environment variables are actually read by the source code.

`server/.env` :

- `DATABASE_URL` : The MongoDB connection string (e.g., MongoDB Atlas).
- `JWT_SECRET` : Used by `auth.controller.ts` and `auth.middleware.ts` to sign and verify session tokens.
- `AI_API_KEY` : The Google Gemini API key used by `ai.service.ts` for the Socratic Tutor.
- `PORT` : (Optional) The port the Express server binds to (defaults to 5000).

No environment variables are strictly required by the frontend client in the current MVP (API URL defaults to relative proxy or `http://localhost:5000` depending on `fetchApi` config).

4. How to Run the Entire Project

Terminal 1 (Backend):

```
cd server
npx tsx src/server.ts
```

Expected output: "Connected to MongoDB Atlas" and "Server is running on port 5000".

Terminal 2 (Frontend):

```
cd client
npm run dev
```

 Expected output: "Ready in Xms".

Verification:

- Open a browser and navigate to `http://localhost:3000`.
- You should see the EduAdapt landing page.
- Navigate to `/login` and authenticate with a seeded user (e.g., `student@example.com` or `teacher@example.com`).

5. Demo Checklist

What is required when showing EduAdapt to someone?

Prerequisites:

- ☐ Backend running (`npx tsx src/server.ts`)
- ☐ Frontend running (`npm run dev`)
- ☐ MongoDB Atlas cluster available and whitelisted.
- ☐ Valid `AI_API_KEY` configured in `server/.env`.
- ☐ `seed.ts` has been executed to populate demo accounts and topics.
- ☐ Browser open to `http://localhost:3000`.

Student Demo Flow:

1. **Landing** → Show conceptual flow.
2. **Login** → Log in as student.
3. **Diagnostic** → Take the initial assessment.
4. **Result** → Observe the mastery gap breakdown.
5. **Dashboard** → Observe the next recommended action (e.g., "PRACTICE: Loops").
6. **Knowledge Map** → Show the prerequisite graph and locked vs. mastered topics.
7. **Learning/Practice** → Enter the recommended topic, submit answers.
8. **Tutor** → Interact with the Socratic AI for a hint.
9. **Dashboard (Updated)** → Show that mastery increased and the recommendation automatically advanced to the next topic (e.g., "Functions").

Teacher Demo Flow:

1. **Teacher Login** → Log in as the teacher.
2. **Dashboard** → Show aggregate metrics and "Students Needing Attention".
3. **Students List** → Select a struggling student.
4. **Student Intelligence** → View their exact conceptual gaps.
5. **Teacher Override** → Manually unlock a locked topic or force a revision.
6. **Verification** → Log back in as the student to prove the teacher's override altered the deterministic path.

6. Complete Project Explanation

1. Project Overview

EduAdapt is a personalized learning engine that merges deterministic knowledge graphs with generative AI tutoring.

2. Problem

Linear curriculums force advanced students to wait and struggling students to drown. AI tutors often hallucinate curriculums or give away answers, failing to enforce rigorous learning objectives.

3. Solution

A hybrid architecture: a rigid, deterministic backend engine that enforces the curriculum and tracks mastery, paired with a generative AI interface that acts purely as an empathetic, Socratic guide restricted by the engine's context.

4. Core Philosophy

The algorithm decides *what* the student learns. The AI helps the student figure out *how* to learn it.

5. Architecture

Client-Server model. Next.js App Router (React 19) for the frontend, Express (Node.js) for the REST API, and MongoDB (Mongoose) for the database.

6. Frontend

A Next.js SPA utilizing Tailwind CSS for styling and `lucide-react` for iconography. Global state is managed by `zustand` (`useAuthStore`).

7. Backend

Express.js REST API utilizing layered architecture: Routes → Controllers → Services → Mongoose Models.

8. Database

MongoDB, connected via Mongoose, relying heavily on `objectId` references to build the knowledge graph (Prerequisites -> Topics -> Objectives).

9. Authentication

JWT-based stateless authentication. Passwords hashed via `bcryptjs` .

10. RBAC

Role-Based Access Control (`Role.STUDENT` vs `Role.TEACHER`), enforced by `auth.middleware.ts` .

11. Student Intelligence

A continuous evaluation loop that calculates `score` , `level` (UNKNOWN, WEAK, DEVELOPING, MASTERED), and `confidence` for every topic.

12. Diagnostic Assessment

A dynamic quiz (currently mocked via seed data) that establishes the initial `MasteryRecord` baseline for a new student.

13. Mastery Engine

Housed in `AdaptiveService`. Calculates mastery based on assessment and practice answers, updating `MasteryRecord`.

14. Learning Objectives

Granular skills beneath a `Topic` (e.g., "Understand for-loop syntax"). Mastery is tracked at both the `Topic` and `Objective` level.

15. Knowledge Gap Engine

Compares current mastery levels against the `Prerequisite` graph to identify what the student is missing before they can advance.

16. Misconception Detection

Future scope / Partially implemented conceptually in `StudentLearningProfile` metadata.

17. Prerequisite Graph

Defined in `content.model.ts` via the `Prerequisite` collection, mapping dependencies between `Topics`.

18. Recommendation Engine

Housed in `AdaptiveService`. Generates a `Recommendation` (e.g., RETEACH, PRACTICE) based on the lowest unlocked node in the `Prerequisite` Graph.

19. Explainable Recommendations

The engine outputs a `reason` string (e.g., "You scored 40% on Loops, practice is recommended") stored directly in the `Recommendation` model and displayed on the Dashboard.

20. Adaptive Learning Path

A sequenced array of `LearningPathNode` documents specific to a student-subject pair.

21. Learning Path States

Nodes are dynamically tagged as `MASTERED`, `CURRENT`, `REVIEW_REQUIRED`, `RECOMMENDED`, or `LOCKED`.

22. Learning Sessions

Tracks time spent and interactions within a specific topic using `LearningSession`.

23. Practice

Students answer multiple-choice `Question` documents.

24. Reassessment

Submitting practice answers triggers `MasteryService.processAssessmentResult()`.

25. Student Dashboard

React component (`dashboard/page.tsx`) that fetches `/analytics/student` to display the active recommendation and mastery levels.

26. Knowledge Map

React component (`dashboard/knowledge-map/page.tsx`) that visually renders the `LearningPathNode` states.

27. Topic Detail

The core learning interface (`topic/[id]/page.tsx`) containing content, practice, and the AI Tutor side-by-side.

28. Socratic AI Tutor

Powered by `@google/genai` in `ai.service.ts` .

29. AI Context Builder

Before sending the prompt to Gemini, `ai.service.ts` injects the student's current mastery level, the topic description, and the specific question they are struggling with.

30. Tutor Modes

`SOCRATIC_HINT` , `EXPLAIN_CONCEPT` , `EXPLAIN_MISTAKE` , `SOCRATIC_CHAT` .

31. Answer Leakage Protection

System instructions strictly prompt the AI: "DO NOT GIVE THE DIRECT ANSWER."

32. Tutor Conversation Persistence

Messages are saved to `TutorConversation` and `TutorMessage` in MongoDB, allowing chat history to persist across page reloads.

33. Gemini Fallback

Error handling in `TutorChat.tsx` displays "Tutor unavailable" if the API key fails or rate limits are hit.

34. Teacher Intelligence

Aggregates student data so teachers can intervene.

35. Teacher Dashboard

Displays class-wide metrics (`teacher/dashboard/page.tsx`).

36. Student Intelligence Drill-down

Allows the teacher to view a specific student's mastery profile (`teacher/students/[id]/page.tsx`).

37. Teacher Intervention

Teachers can log `TeacherIntervention` records (e.g., "Needs 1-on-1 support").

38. Teacher Override

Teachers can create a `TeacherOverride` (e.g., `UNLOCK_TOPIC`), which is explicitly respected by the `AdaptiveService` during path generation.

39. Teacher Content Management

Deferred. (Content is currently seeded via script).

40. Security

JWT Http headers, Helmet (backend), CORS.

41. IDOR Protection

`TeacherProfile` contains a `students` array. The `teacher.controller.ts` restricts queries so teachers can only view their assigned students.

42. Input Validation

Incoming requests (auth, answers) are validated manually in controllers.

43. Database Indexes

Compound indexes exist on `MasteryRecord {student: 1, topic: 1}`, `Recommendation`, and `LearningPathNode` for query optimization.

44. API Architecture

Standard RESTful JSON endpoints (e.g., `GET /analytics/student`, `POST /tutor/message`).

45. Frontend Routes

Next.js App Router paradigm (`app/page.tsx`, `app/dashboard/page.tsx`).

46. Backend Routes

Express routers defined in `server/src/app.ts` (`/api/auth`, `/api/learning`, `/api/teacher`).

47. Testing Architecture

Standalone TypeScript execution scripts (`tsx src/utils/test-*.ts`) utilizing Mongoose directly against the live database for rapid E2E verification.

48. Integration Tests

`test-loop.ts` verifies that submitting a failing answer legitimately alters the database's `Recommendation`.

49. E2E Test

`test-e2e.ts` runs a master simulation of Student Login -> Diagnostic -> Teacher Override -> Student Path Verification.

50. UX/Productization

Implemented standard `LoadingState`, `EmptyState`, and `ErrorState` UI components to prevent raw JSON/stack traces from leaking to the user.

51. Responsive Design

Tailwind `md:`, `lg:` prefixes ensure the Dashboard and Knowledge Map adapt to mobile screens.

52. Accessibility

Semantic HTML and high-contrast color palettes (Red/Off-white) utilized.

53. Error/Loading/Empty States

Verified across all 8 major frontend routes.

54. Current Technical Debt

Next.js Server Components are underutilized; the frontend relies heavily on `use client` and `useEffect` fetching (SPA pattern).

55. Deferred Features

Content Management UI, advanced Gamification.

56. Deployment Status

Deferred (Currently Localhost MVP).

57. SIH Demo Flow

Verified to work cleanly via `npm run dev`.

58. Future Scope

Predictive dropout analytics, real-time audio/video Socratic tutoring via WebRTC.

7. Database Documentation

The following Mongoose models are explicitly implemented and verified in the source code:

User & Auth (`user.model.ts`)

- `User` : Core authentication entity (email, passwordHash, role).
- `StudentProfile` : Metadata for students (learningGoal, studyTimeAvailability).
- `TeacherProfile` : Metadata for teachers (contains `students: [ObjectId]` array for IDOR protection).



Content (`content.model.ts`)

- `Subject` : High-level domain (e.g., "Computer Science").
- `Topic` : Primary learning unit (e.g., "Loops").
- `LearningObjective` : Granular skill inside a Topic.
- `Prerequisite` : Directed edge defining dependencies (e.g., Topic A must be mastered before Topic B).

Assessment (`assessment.model.ts`)

- `Question` : Multiple-choice question linked to a Topic/Objective.
- `Assessment` : A collection of questions (e.g., the Diagnostic).
- `AssessmentAttempt` : A tracking record of a student starting an assessment.
- `Answer` : A student's submitted answer to a specific question.

Intelligence (`intelligence.model.ts`)

- `MasteryRecord` : The core metric. Tracks a student's `score` (0-1) and `level` (WEAK, MASTERED) for a specific Topic.
- `StudentLearningProfile` : Aggregate metadata about the student's learning speed and preferences.
- `Recommendation` : The deterministic engine's output (e.g., `actionType: PRACTICE` , `recommendedTopic`).

Learning (`learning.model.ts`)

- `LearningPath` : A container for a student's sequence through a Subject.
- `LearningPathNode` : A specific stop on the path, containing a `state` (LOCKED, CURRENT, MASTERED).
- `LearningSession` : Time-tracking for when a student is actively studying a Topic.
- `RevisionItem` : Scheduled spaced-repetition tasks.

Teacher (`teacher.model.ts`)

- `TeacherIntervention` : A log of a teacher's manual action.
- `TeacherOverride` : A hard override (e.g., `UNLOCK_TOPIC`) that forces the `AdaptiveService` to ignore the prerequisite graph.

Tutor (`tutor.model.ts`)

- `TutorConversation` : A chat session linked to a specific Topic and Student.
- `TutorMessage` : Individual messages (user or model) stored for persistence.

8. Adaptive Engine Documentation

The Actual Flow:

1. **Assessment:** Student submits `Answers` via `POST /api/assessment/:id/submit` (`assessment.controller.ts`).
2. **Mastery Calculation:** `AdaptiveService.updateMastery()` recalculates the student's moving average score and updates the `MasteryRecord` .
3. **Graph Evaluation:** `AdaptiveService.generateRecommendation()` queries the `Prerequisite` collection. It finds all Topics where prerequisites are `MASTERED` but the Topic itself is `WEAK` or `UNKNOWN` .
4. **Recommendation Output:** The lowest-ordered available Topic is selected. A `Recommendation` document is written to the DB.

- 
5. **Path Regeneration:** `AdaptiveService.updateLearningPath()` is triggered, updating the `LearningPathNode` states (e.g., changing a node from `LOCKED` to `CURRENT`).

6. **Delivery:** The Frontend Dashboard fetches the active `Recommendation` and directs the student to practice.

9. Master Example (System Verification)

Scenario verified by `test-Loop.ts`:

Initial State: Student A has a `MasteryRecord` for "Loops" at 48% (WEAK). The active `Recommendation` is `PRACTICE: Loops`.

Action: Student A practices "Loops" and submits correct answers.

After successful practice: `AdaptiveService` recalculates the `MasteryRecord` for "Loops", raising it to 83.6% (MASTERED).

Engine Response: Because "Loops" is now MASTERED, the prerequisite for "Functions" is satisfied. The engine deletes the old recommendation and generates a new `Recommendation` for `LEARN: Functions`. The Knowledge Map automatically unlocks "Functions".

This proves genuine programmatic adaptation without hallucination.

10. AI Tutor Documentation

EduAdapt integrates Google Gemini strictly as a conversational guide, **not** as a curriculum decider.

- **Context Injection:** When a student sends a message, `tutor.controller.ts` fetches the student's `MasteryRecord` and the `Topic` description, prepending it as a System Prompt to Gemini.
- **Tutor Modes:** Supports `SOCRATIC_HINT`, `EXPLAIN_CONCEPT`, `EXPLAIN_MISTAKE`, and `SOCRATIC_CHAT`.
- **Leakage Protection:** The prompt explicitly commands Gemini: *"Do not give the direct answer to the multiple choice question."*
- **Persistence:** Managed via `TutorMessage` in MongoDB, ensuring context isn't lost on page refresh.

11. Teacher Documentation

EduAdapt preserves human authority.

- **RBAC:** Teachers authenticate and receive a JWT with `role: TEACHER`.
- **Dashboard:** `teacher/dashboard/page.tsx` pulls aggregate class data.
- **Overrides:** If the AI engine stubbornly locks a topic, the Teacher can issue a `TeacherOverride` (`UNLOCK_TOPIC`). The `AdaptiveService` explicitly checks for overrides before locking nodes, proving that Human-in-the-Loop authority supersedes the deterministic graph.

12. Security Documentation

Verified Measures:

- **Authentication:** JWT tokens required for all `/api/learning` , `/api/intelligence` , `/api/teacher` , and `/api/tutor` routes.
- **Password Protection:** Stored exclusively as `bcrypt` hashes.
- **IDOR Protection:** `teacher.controller.ts` explicitly filters student queries using `TeacherProfile.students { $in: [...] }` . A teacher cannot fetch data for unassigned students.
- **Secret Handling:** Keys are in `.env` , ignored by `.gitignore` .

13. Testing Documentation

Integration tests run directly via Node `tsx` against the live MongoDB instance:

- `test-loop.ts` : Verifies the core Adaptive Engine. Asserts that submitting passing scores flips Mastery to MASTERED and updates the Recommendation. *(Status: PASS)*
- `test-diagnostic.ts` : Verifies Assessment evaluation logic. *(Status: PASS)*
- `test-tutor.ts` : Verifies Gemini API connectivity, prompt formatting, and DB persistence. *(Status: PASS)*
- `test-teacher.ts` : Verifies RBAC logic and Teacher Override functionality against the AdaptiveService. *(Status: PASS)*
- `test-e2e.ts` : Master test linking all sub-systems. *(Status: PASS)*

14. Project Phase History

- **Phase 1 (Foundation):** Express/Next.js setup. (VERIFIED)
- **Phase 2 (Database/Domain):** Mongoose schemas (content, learning). (VERIFIED)
- **Phase 3 (Auth/Onboarding):** JWT, Login/Register UI. (VERIFIED)
- **Phase 4 (Diagnostic + Adaptive Engine):** Prerequisite engine, Mastery math, Knowledge Map. (VERIFIED)
- **Phase 5 (Socratic AI Tutor):** Gemini integration, Chat UI. (VERIFIED)
- **Phase 6 (Teacher Intelligence):** Overrides, Dashboards. (VERIFIED)
- **Phase 7 (Production Hardening):** IDOR fixes, Indexing, E2E Testing. (VERIFIED)
- **Phase 8 (Productization/UX):** Empty/Error states, Navigation Bar. (VERIFIED)

Deployment is DEFERRED.

15. Required Project Information

Project Name: EduAdapt **Project Title:** EduAdapt: Deterministic AI-Powered Adaptive Learning **Project Description:** A personalized learning platform that merges deterministic curriculum knowledge graphs with generative AI tutoring to provide students with tailored, explainable learning paths while empowering teachers with classroom intelligence. **Problem Statement:** Traditional linear curriculums fail to accommodate individual learning paces, while purely generative AI tutors hallucinate curriculums and fail to reliably enforce rigorous pedagogical objectives. **Your solution:** A hybrid architecture where a deterministic backend engine securely calculates mastery and generates prerequisite-based learning paths, augmented by a bounded Socratic AI tutor that guides students without revealing answers. **Uniqueness & Innovation:** The strict architectural separation of "Curriculum Intelligence" (Deterministic math) and "Conversational Intelligence" (Generative AI), preventing hallucination-induced learning dead-ends, paired with explicit Human-in-the-Loop overrides for teachers.

16. Technology Stack

Implemented in Repository:

- **Languages:** TypeScript, JavaScript
- **Frontend:** Next.js (App Router), React, Tailwind CSS, Zustand, Lucide React
- **Backend:** Node.js, Express.js
- **Database:** MongoDB (Atlas), Mongoose (ODM)
- **Authentication:** JWT, bcryptjs
- **AI Integration:** Google Gemini (@google/genai SDK)
- **Security:** Helmet, CORS

17. What Not To Claim

EduAdapt **DOES NOT** currently implement:

- Subtopic-level granularity (Model not found in code).
- `PracticeAttempt` , `ProgressRecord` , `Milestone` , `Notification` , Or `AuditLog` models (Deferred/Future scope).
- Live WebRTC / Camera Emotion Detection.
- Real production deployment (Currently local/development only).
- Scientific/pedagogical validation (Only functional software verification).

18. Final Project Status

Component	Status	Notes
Adaptive Engine	IMPLEMENTED	Verified via <code>test-loop.ts</code> .
Diagnostic	IMPLEMENTED	Seeds initial <code>MasteryRecord</code> .
Mastery	IMPLEMENTED	Mathematical rolling average in <code>AdaptiveService</code> .
Learning Objectives	IMPLEMENTED	Handled in schema, though UI focuses heavily on Topics.
Knowledge Gaps	IMPLEMENTED	Graph traversal via <code>Prerequisite</code> edges.
Recommendations	IMPLEMENTED	Explains "Why this topic?".
Learning Path	IMPLEMENTED	<code>LearningPathNode</code> dynamically tracks states.
Knowledge Map	IMPLEMENTED	Visual UI connected to Path API.
Socratic Tutor	IMPLEMENTED	Gemini integrated via <code>ai.service.ts</code> .
Teacher Override	IMPLEMENTED	Verified to break graph locks via <code>test-teacher.ts</code> .
RBAC / Security	IMPLEMENTED	IDOR protection active.
Testing	IMPLEMENTED	5 Custom Integration Suites passing.
UX Polish	IMPLEMENTED	Unified Loading/Error states across routes.
Deployment	DEFERRED	Localhost only.



Document Generated via Full Repository Inspection.